

Aichestra 기술 스택 및 동작 설계도

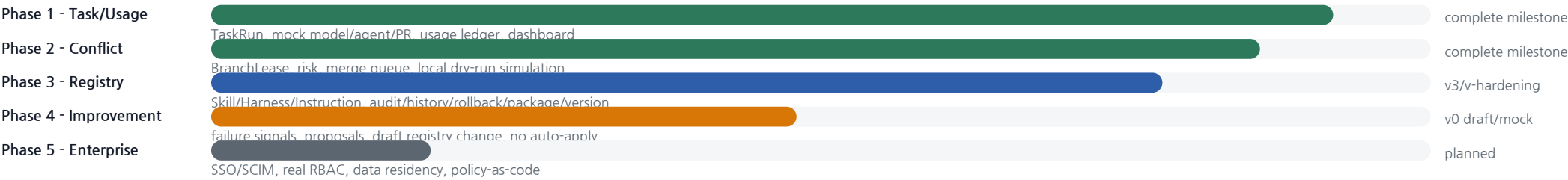
LLM/Agent 협업을 위한 중앙 오케스트레이션 제어면 - architecture, data flow, registry, conflict, improvement blueprint



설계 기준

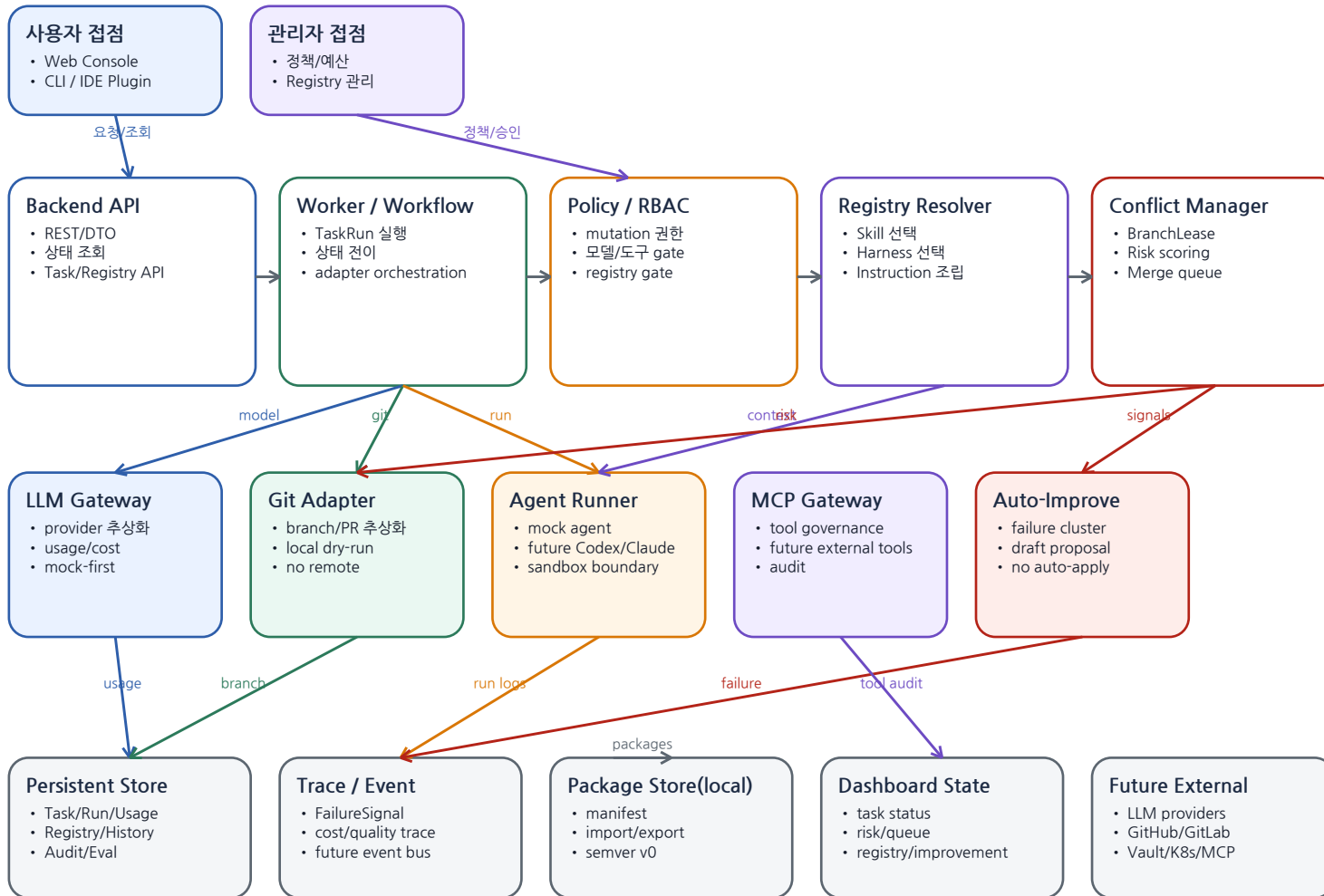
이 문서는 현재까지 정의한 Aichestra 프로젝트를 하나의 기술 설계도로 정리한다. 실제 provider 연동을 붙이기 전의 mock-first 제어면을 기준으로 하며, 각 컴포넌트가 어떤 기능을 담당하고 다른 요소에 어떤 영향을 주는지 연결 관계를 중심으로 설명한다.

Phase 진행 상태(현재 마일스톤 기준)



현재 완성도 해석: 내부 mock-first 프로토타입으로는 상당히 진전되었지만, 실제 제품 운영에는 real LLM Gateway, real Git provider, runner/sandbox, secrets, deployment, production RBAC가 남아 있다.

전체 아키텍처: 중앙 제어면이 모델, 코드, 레지스트리, 충돌, 개선 제안을 조율

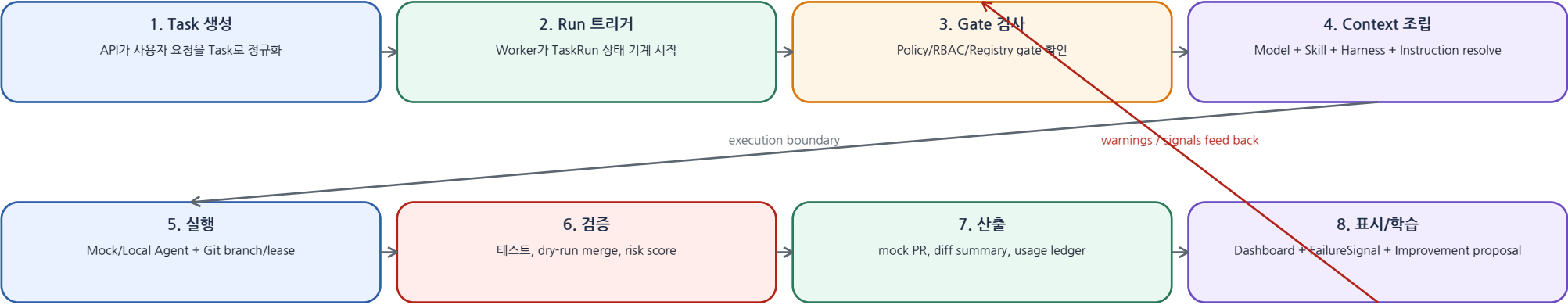


읽는 방법: 사용자 요청은 Backend API로 들어오고, Worker/Workflow가 TaskRun을 진행한다. Policy/RBAC가 모든 mutation과 실행을 gate하고, Registry Resolver가 Skill/Harness/Instruction을 선택한다. Conflict Manager는 branch/lease/risk/queue를 관리하며, 결과는 store/trace/dashboard/auto-improvement loop로 흘러간다.

1. 기술 스택 맵과 영향 관계

계층	추천/현재 기술	맡는 기능	다른 요소에 주는 영향
User / UI	Next.js Web, CLI, IDE Plugin	작업 생성, 상태 확인, Registry/Conflict/Improvement 확인	API DTO 안정성이 UX와 자동화 스크립트의 계약이 된다.
API	TypeScript/NestJS 또는 Go API	Task/Registry/Conflict/Improvement endpoint, validation, DTO mapping	orchestration 로직을 직접 품지 않고 worker/service boundary를 호출해야 한다.
Worker / Workflow	현재 mock worker, 미래 Temporal/Argo	TaskRun 상태 전이, adapter 호출, retry/handoff	모델/Registry/Git/Conflict의 결과를 하나의 실행 단위로 묶는다.
Policy / RBAC	mock authorizer, 미래 OPA/Cedar/SSO	registry mutation, model/tool/merge/approval gate	prompt 지시문보다 강한 강제 규칙 계층이다.
LLM Gateway	mock gateway, 미래 LiteLLM/자체 proxy	model routing, cost, usage, trace, redaction	비용을 taskRunId/skill/harness에 귀속시켜 운영 최적화가 가능해진다.
Git Adapter	mock/local git, 미래 GitHub/GitLab App	branch, PR, dry-run merge, lease	Conflict Manager의 데이터 소스이자 agent 실행 산출물의 외부 접점이다.
Agent Runner	mock runner, 미래 Codex/Claude/Aider adapter	Skill/Harness/Instruction 기반 실행	runner가 바뀌어도 TaskRun/Registry/Gateway 계약은 유지되어야 한다.
Registry	Skill/Harness/Instruction repositories	version, approval, eval, checksum, history, rollback, package	agent 행동과 실행 환경을 중앙에서 재현 가능하게 만든다.
Conflict Manager	BranchLease, ConflictRisk, MergeQueue	파일 overlap, local simulation, queue recommendation	동시 AI 작업의 merge 순서와 review 필요성을 결정한다.
Auto-improvement	FailureSignal, Cluster, Proposal, DraftChange	실패 신호를 개선 후보와 초안으로 변환	v0에서는 active registry를 직접 바꾸지 않고 draft만 생성한다.
Storage / Trace	Postgres 후보, local/in-memory repos	Task, Usage, Registry, Audit, History, Eval, Proposals	모든 분석은 taskId/taskRunId와 revision/audit key에 의존한다.

TaskRun 실행 흐름: 요청이 추적 가능한 작업 산출물로 바뀌는 과정



항상 남기는 추적 키

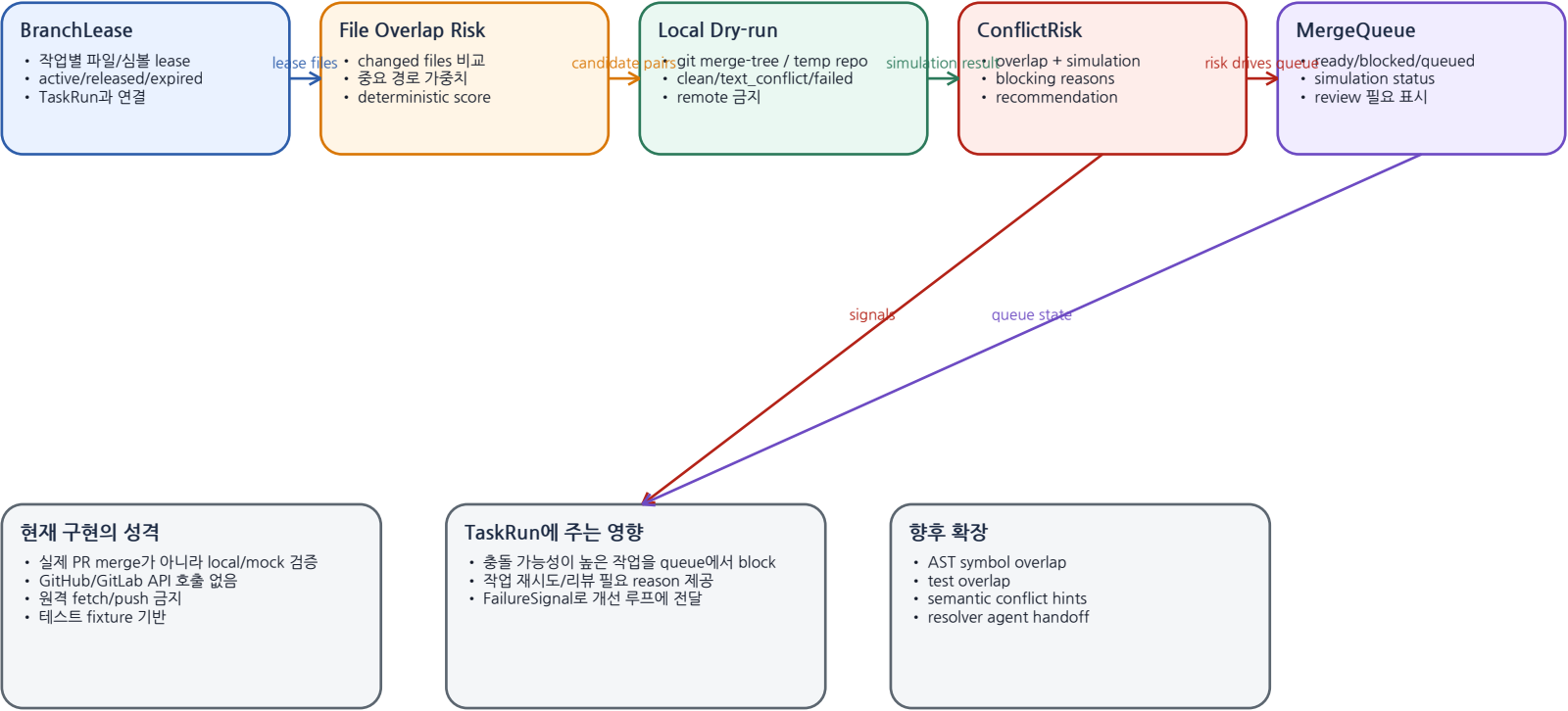
taskId, taskRunId, actorId, modelRef, skillRefs, harnessRef, instructionRefs, branchLeaseId, conflictRiskId, usageLedgerId, auditLogId
이 키들이 있어야 나중에 비용/품질/충돌/Registry 변경 원인을 연결해서 분석할 수 있다.

핵심 영향: TaskRun이 모든 요소를 연결하는 중심 레코드다. 모델 비용, Skill/Harness/Instruction 선택, branch lease, conflict risk, failure signal, improvement proposal이 모두 TaskRun을 통해 추적 가능해야 한다.

TaskRun 상태 설계 체크리스트

상태/단계	필요한 데이터	다음 컴포넌트에 끼치는 영향
created / policy_checked	actor, repo, requested task, policy decision	권한 거부/승인에 따라 worker 진입 여부 결정
registry_resolved	selectedSkillRefs, harnessRef, instructionRefs, warnings	agent prompt/context와 실행 환경을 결정
branch_prepared	branchLeaseId, target files, local branch info	Conflict Manager가 active lease graph를 갱신
agent_running / diff_generated	changedFiles, diffSummary, test hints	usage ledger와 conflict risk 재계산의 입력
tests_passed / pr_created	mock PR URL, test result, risk status	dashboard 표시와 merge queue 추천에 영향
completed / failed / blocked	result, failure signals, audit refs	auto-improvement 후보와 registry 개선 제안의 출발점

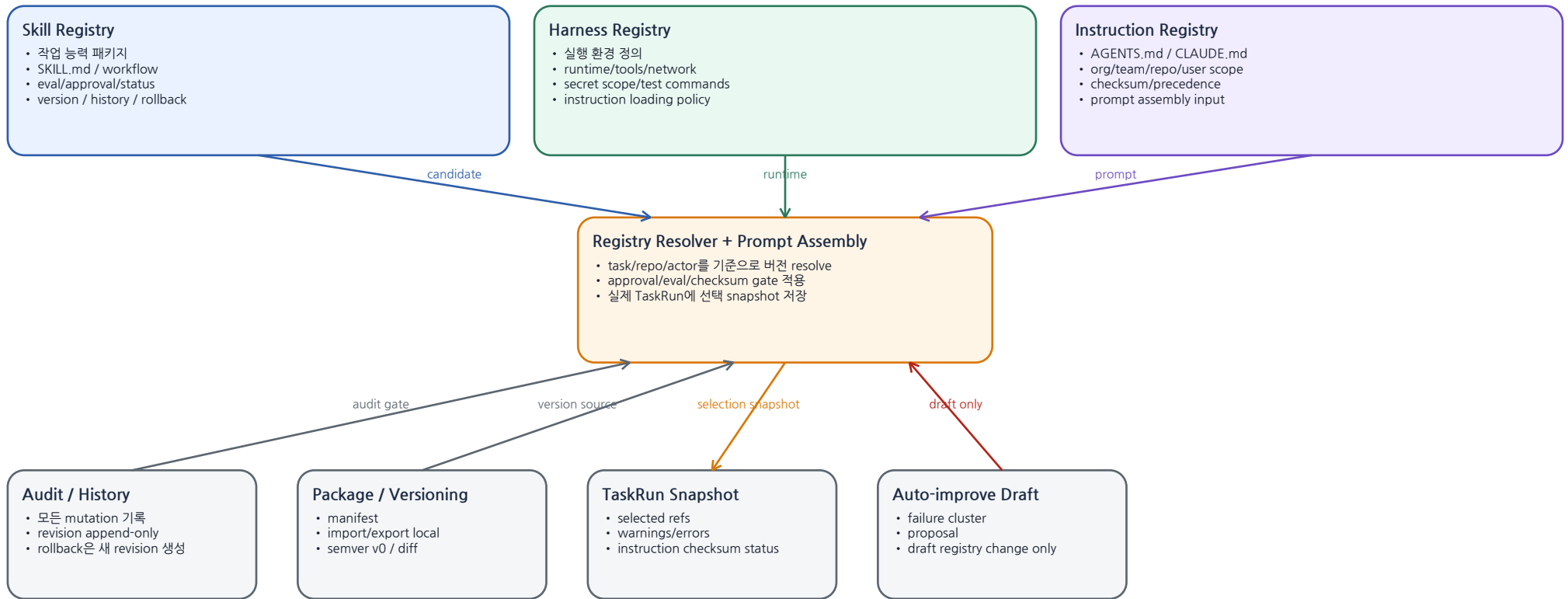
Conflict Manager: 여러 AI 작업을 동시에 돌릴 때 충돌을 예측하고 queue를 제어



Conflict Manager는 실제 merge를 수행하는 모듈이 아니라, 동시 AI 작업이 서로 부딪힐 가능성을 계산하고 queue/review 결정을 돕는 제어면이다. v1 기준으로 local dry-run simulation은 remote fetch/push 없이 수행되어야 한다.

요소	입력	출력	주의점
BranchLease	taskRunId, branch, changed/expected files	active/released/expired lease	같은 repo에서 동시 실행되는 AI 작업의 점유 범위를 보여준다.
ConflictRisk	lease overlap, risk path, dry-run result	riskScore, reasons, recommendation	text conflict는 high risk, unavailable은 warning으로 처리한다.
MergeSimulator	local repo path, base/source refs	clean/text_conflict/failed/unavailable	원본 working tree를 변경하거나 remote 작업을 하면 안 된다.
MergeQueueEntry	PR/taskRun/risk/simulation	ready/blocked/queued/review-needed	자동 merge보다 review routing과 순서 결정에 집중한다.

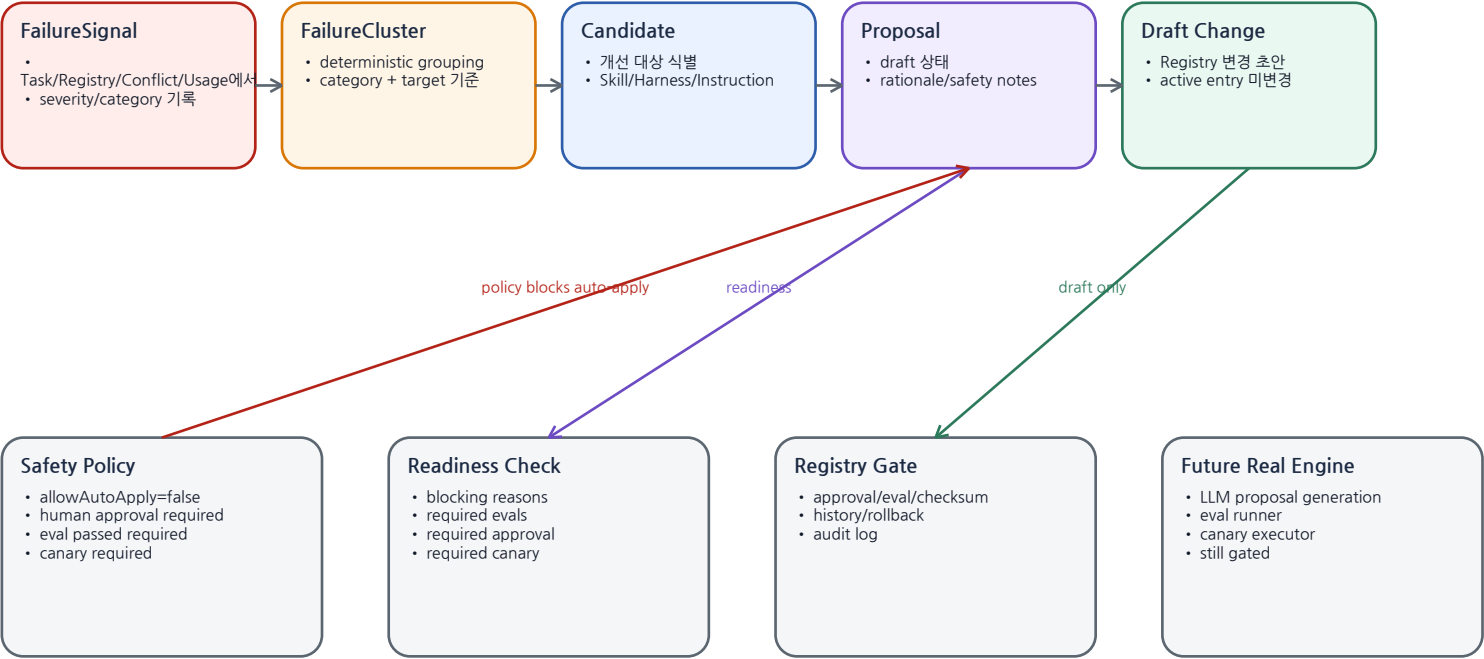
Registry / Prompt Context 설계: 무엇을 실행하고 무엇을 주입할지 분리



핵심 분리: Skill은 “무엇을 어떻게 수행할지”, Harness는 “어떤 환경과 권한으로 실행할지”, InstructionArtifact는 “어떤 지속 지시문을 주입할지”를 나타낸다. 세 개를 합치면 실패 원인과 안전 정책을 추적하기 어려워진다.

Registry 요소	상태/게이트	영향
SkillPackage	status + approvalStatus + evalStatus + version	작업 절차와 모델/agent 호환성을 결정한다. 잘못 선택되면 비용/품질에 직접 영향.
HarnessDefinition	runtime, tools, network, secret scopes, test commands	Agent가 실행되는 환경을 고정한다. 재현성, 보안, 테스트 안정성에 영향.
InstructionArtifact	type/scope/precedence/checksum	AGENTS.md, CLAUDE.md 같은 prompt context를 관리한다. 행동 편차와 정책 충돌을 줄인다.
RegistryResolver	version range, default selection, gates	TaskRun에 실제 선택 snapshot을 남긴다. warning/error가 failure signal이 된다.
History/Audit/Rollback	append-only revision, mutation audit	Phase 4 개선 제안이 안전하게 검토/되돌리기 가능하도록 만든다.
Package/Versioning	manifest, import/export, semver v0, diff	Registry 자산을 이동/비교/묶음 배포할 수 있게 한다.

Auto-improvement v0: 실패 신호를 개선 제안으로 바꾸되 자동 적용은 금지

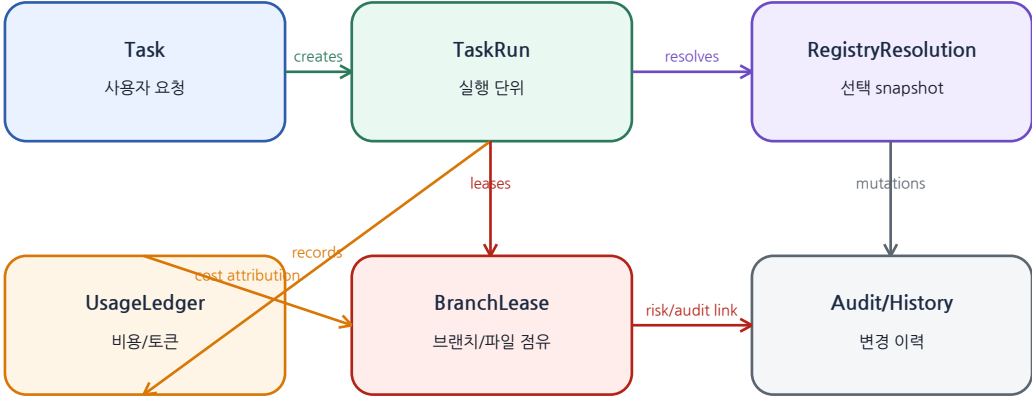


중요 안전 규칙: v0는 제안과 초안만 만들며, active Registry 항목을 자동 승인/적용/활성화하지 않는다.

Auto-improvement v0의 목적은 “자동 수정”이 아니라 “안전하게 검토 가능한 개선 초안 생성”이다. 실제 LLM이나 active registry mutation 없이 deterministic mock engine으로 상태 모델과 safety gate를 먼저 검증한다.

단계	만드는 객체	active Registry 변경 여부	다음 단계에 미치는 영향
신호 수집	FailureSignal	없음	Task 실패, resolver warning, conflict, cost spike를 기록
군집화	FailureCluster	없음	반복 문제를 targetKind/targetRef 기준으로 묶음
후보 생성	ImprovementCandidate	없음	Skill/Harness/Instruction 중 개선 대상을 식별
제안 생성	ImprovementProposal	없음	draft 상태의 개선 rationale, safety notes 작성
초안 변경	DraftRegistryChange	없음	검토/승인/eval/canary 전까지 active entry에 반영 금지
준비성 평가	ProposalReadiness	없음	approval/eval/canary/policy blocking reasons 반환

주요 도메인 관계도: 상태/비용/품질/개선 원인을 연결하는 키

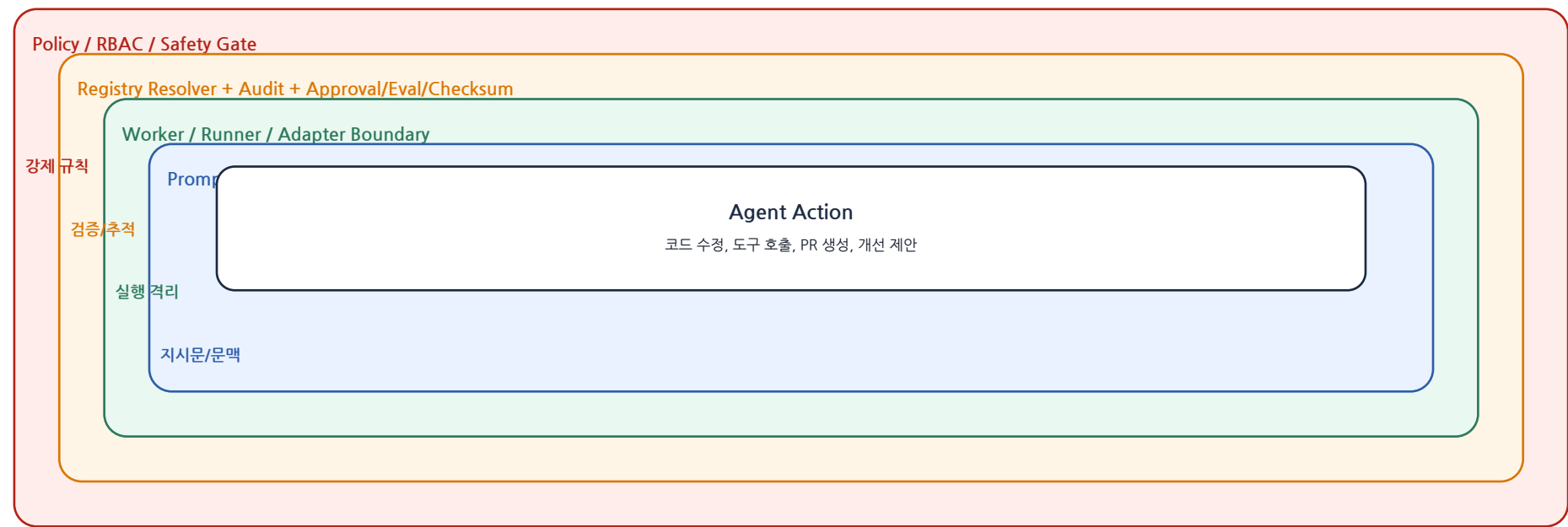


핵심 추적 원칙

모든 실행 결과는 taskId/taskRunId에 귀속되어야 한다. Registry 선택, conflict risk, usage ledger, audit log가 서로 같은 실행 단위를 가리켜야 원인 분석이 가능하다.

이 관계가 유지되어야 실제 provider를 붙인 후에도 원인 분석이 가능하다. 예: 특정 Skill 버전이 특정 Harness에서 비용이 높고 conflict risk를 자주 만들었다는 식의 분석이 가능해진다.

보안/정책 경계: prompt 지시문은 정책을 대체하지 않는다



CLAUDE.md/AGENTS.md는 Instruction Layer이며, prod 접근 금지/권한/승인 같은 제약은 Policy/Runtime/MCP Gateway에서 강제해야 한다.

2. API / 데이터 흐름 요약

영역	대표 API/객체	동작 방식	영향
Task	POST /tasks, POST /tasks/:id/run	Task 생성 후 Worker가 TaskRun을 진행	실행 단위와 추적 키를 만든다.
Usage	GET /usage?taskId=...	LLM/agent mock usage를 taskRunId에 귀속	비용-품질-Registry 분석의 기반.
Conflict	/conflict/risks, /merge-queue, /simulation	lease/risk/queue/local simulation 표시	동시 AI 작업의 충돌 회피.
Registry	/registry/skills, /harnesses, /instructions	CRUD, status, approval/eval, checksum	agent 실행 문맥과 환경의 중앙 관리.
Registry Ops	/registry/audit, /history, /rollback	revision/audit/rollback 조회 및 실행	안전한 변경 관리와 복구.
Packaging	/registry/packages/import/export/diff	local manifest 기반 import/export/diff	Registry artifact 이동과 비교.
Improvement	/improvement/*	failure cluster, proposal, draft change	자동 개선의 draft-only 기반.

3. 실제 연동을 붙일 때의 권장 순서

순서	연동 대상	왜 이 순서인가	주의점
1	Persistent DB(Postgres)	현재 in-memory/local repo를 운영 저장소로 바꾸는 첫 단계	Repository interface를 유지하고 migration/test fixture를 분리한다.
2	Real Git Adapter v0	Conflict/Task/PR의 외부 접점	최소 권한 GitHub App, read/write scope 분리, remote merge 금지로 시작.
3	LLM Gateway v0	모델 호출과 비용의 실제 관문	LiteLLM/자체 proxy, virtual key, budget, redaction, usage ledger 연계.
4	Local Agent Runner v0	실제 Codex/Claude/Aider 실행	sandbox, timeout, no secret logging, deterministic validation command.
5	Secrets/Runtime/Sandbox	안전한 실행을 위한 토대	Vault/KMS, Docker/K8s/Firecracker, network allowlist.
6	MCP Gateway v0	외부 도구 사용 governance	tool-level permission, audit, human approval gate.

4. 설계상 가장 중요한 운영 원칙

원칙	설명
Mock-first, provider-later	실제 API를 먼저 붙이지 않고 도메인/상태/정책/테스트를 먼저 고정한다.
TaskRun 중심 추적	모든 비용, Registry 선택, branch lease, conflict risk, proposal은 taskRunId로 연결한다.
Policy는 prompt보다 강하다	AGENTS.md/CLAUDE.md는 지시문일 뿐이며 권한/보안/승인은 Policy/Runtime에서 강제한다.
Registry는 운영 artifact다	Skill/Harness/Instruction은 version, approval, eval, checksum, audit, rollback이 있어야 한다.
Auto-improvement는 draft-only로 시작한다	v0에서는 active registry를 자동 변경하지 않는다. proposal/readiness/safety gate가 먼저다.
실제 연동은 interface 뒤에 붙인다	LLM, Git, MCP, artifact, secrets, runtime은 모두 adapter/repository/gateway boundary를 통과한다.

현재 설계의 의미

Aichestra는 지금 단계에서 “실제 외부 연동 없는 mock-first AgentOps control plane”으로 볼 수 있다. Task orchestration, conflict management, registry governance, draft-only improvement loop가 연결되어 있으므로, 다음 과제는 이 경계를 유지한 채 real Git/LLM/runner/security integration을 하나씩 붙이는 것이다.

산출물 구성

이 PDF는 공유/검토용 설계도이며, 함께 생성된 Markdown 파일은 Mermaid 기반 원문/수정용 설계 문서로 사용할 수 있다.